TD 2

La POO : pensez en objets !

Vers le bac en **TD dirigé** : simuler un programme, compléter puis écrire des méthodes, faire interagir plusieurs objets.

Nom, prénom ..... Classe ..... Type bac

À VOS SMARTPHONES !

La vraie loi de Moore

1946 EXERCICE 1 — l'ENIAC · la loi de Moore
1969 EXERCICE 3 — ARPANET · le premier message
1971 EXERCICE 2 — Intel 4004 · les microprocesseurs
1974 EXERCICE 4 — Cerf et Kahn · TCP/IP

Mode d'emploi

Ce TD dirigé prolonge le TD 1 pour consolider les concepts fondamentaux de la POO, avec des questions proches de celles du baccalauréat. Vous y croiserez des figures marquantes de l'histoire de l'informatique, de l'ENIAC à Internet.

Rappel : le cours utile **Coup de pouce** : une aide si vous bloquez **Méthode** : la démarche pas à pas

EXERCICE 1

Le premier ordinateur, l'ENIAC : la loi de Moore 1946

Création, modification d'attribut et simulation

Cet exercice porte sur la programmation orientée objet, les boucles et les puissances de 2.

Le premier ordinateur électronique, l'**ENIAC**, marquait une révolution dans la puissance de calcul : il pouvait effectuer environ **5 000 additions par seconde**. La puissance d'un ordinateur est souvent mesurée par sa fréquence de processeur, qui correspond au nombre d'opérations primaires qu'il peut réaliser en une seconde.

Plus tard, en 1965, **Gordon Moore** (cofondateur d'Intel) prédit que la puissance des puces doublerait approximativement tous les deux ans[1]. Cette observation est devenue la célèbre « **loi de Moore** ». Nous allons simuler cette croissance en considérant que la puissance double tous les deux ans.

Ordinateur
nom : str annee : int puissance : nombre
augmenter_puissance(facteur)

Rappel de cours

`self.puissance *= facteur` équivaut à `self.puissance = self.puissance * facteur` : la méthode **modifie** l'attribut de l'objet, elle ne renvoie rien.

```python
class Ordinateur:
    def __init__(self, nom, annee, puissance):
        self.nom = nom
        self.annee = annee
        self.puissance = puissance

    def augmenter_puissance(self, facteur):
        self.puissance *= facteur
```

PARTIE A — Simuler un programme

1 Que vaut `mon_vieux_coucou.puissance` à la fin du code suivant ?

```python
# Créez l'objet 'eniac' avec les paramètres : "ENIAC", 1946, 5000
mon_vieux_coucou = Ordinateur("ENIAC", 1946, 5000)
for i in range(10):
    mon_vieux_coucou.augmenter_puissance(2.5)
```

Coup de pouce Combien de fois la méthode est-elle appelée ? Écrivez d'abord le résultat sous la forme $5\,000 \times 2{,}5^{\cdots}$ avant de calculer.

Brouillon

[1] Pour connaître la vraie loi de Moore : fr.wikipedia.org/wiki/Loi_de_Moore (QR code en haut de la page 1).

PARTIE B Combien d'années pour rattraper un ordinateur moderne ?

Un ordinateur moderne a une puissance arbitraire de 2^{28} (soit environ 268 millions d'unités). On suppose toujours que la puissance **double tous les deux ans**.

2 Combien d'années faut-il à l'ENIAC (d'une puissance de 5 000 opérations par seconde) pour atteindre cette puissance ?

> **Coup de pouce** Cherchez le plus petit nombre de doublements n tel que $5\,000 \times 2^n \geq 2^{28}$. Aide : $2^{28} \div 5\,000 \approx 53\,687$; $2^{15} = 32\,768$; $2^{16} = 65\,536$.

3 Compléter le code ci-dessous permettant de réaliser ce calcul.

```
# Créez l'objet 'eniac' avec les paramètres : "ENIAC", 1946, 5000
eniac = ________________________________________________
nb_annees = 0
# La boucle doit continuer tant que la puissance de l'eniac est inférieure à la cible
while ____________________ < 2**28:
    nb_annees ____________
    # Augmentez la puissance de l'ENIAC par un facteur 2 (sans utiliser de =)
    ________________________________________________

# Afficher le nombre d'années
________________________________________________
```

4 Cela vérifie-t-il la réalité ? Aidez-vous de l'année à laquelle l'ENIAC aurait atteint 2^{28} selon ce modèle.

PARTIE C Et aujourd'hui ?

5 Selon ce modèle (puissance doublée tous les 2 ans), si un ordinateur a aujourd'hui une puissance de 2^{28}, quelle puissance (en exposant de 2) aura-t-il dans dix ans ? Détaillez le calcul.

6 L'augmentation de la fréquence des processeurs est-elle toujours possible à ce rythme aujourd'hui ? Expliquez brièvement pourquoi (considérations physiques).

> **Coup de pouce** Pensez à la chaleur dégagée par les transistors et à leur taille, qui approche aujourd'hui celle de quelques atomes.

Brouillon

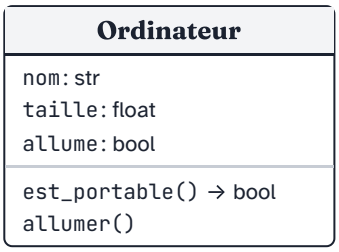
EXERCICE 2

L'ère des microprocesseurs années 1970

Instanciation, portée et écriture de méthode

Cet exercice porte sur la programmation orientée objet : constructeur, méthodes, et différence entre méthode et fonction.

L'invention des microprocesseurs (comme l'Intel 4004 en 1971) a permis de créer des ordinateurs beaucoup plus petits et personnels. Ces machines, bien que distinctes, utilisent toutes la même structure de base : la classe `Ordinateur`.




```python
class Ordinateur:
    def __init__(self, nom, taille):
        self.nom = nom
        self.taille = taille  # Taille en mètres cubes (arbitraire)
        self.allume = False   # État par défaut
        print(f"DEBUG : Constructeur appelé pour {self.nom}")

    def est_portable(self):
        # Un ordinateur est portable si sa taille est inférieure ou égale à 1.0 m³
        ---------------------------------------------------------------------------------------------------- ◂ question 4

    def allumer(self):
        # Allume l'ordinateur et affiche l'information
        # exemple : ENIAC est allumé.
        ---------------------------------------------------------------------------------------------------- ◂ question 3
        ----------------------------------------------------------------------------------------------------

def eteindre(machine):
    # C'est une erreur de POO d'utiliser cette fonction pour éteindre une machine !
    machine.allume = False
    print(machine.nom + " est éteint(e).")
```

PARTIE A — Instancier

Rappel de cours

Instancier, c'est appeler la classe comme une fonction : `objet = Classe(arguments)`. Les arguments sont transmis à `__init__` ; `self` est fourni automatiquement par Python.

1 Écrire le code Python permettant de créer deux objets de la classe `Ordinateur` avec les paramètres suivants :

- `apple2` : nommé `"Apple II"` et de taille 0,8 m³ ;
- `desktop` : nommé `"Desktop 95"` et de taille 2,5 m³.

2 Si on exécute le code Python de la question précédente, qu'affiche le terminal ? Pourquoi ?

PARTIE B Écrire des méthodes

3 Compléter la méthode `allumer` (lignes 15 et 16 du code).

4 Compléter la méthode `est_portable` (ligne 10 du code).

> **Méthode — écrire une méthode**
> 1. Quel attribut faut-il **lire** ou **modifier** ? On y accède avec `self.` …
> 2. La méthode doit-elle **renvoyer** une valeur (`return`) ou **modifier** l'objet ?
> 3. Une comparaison comme `a <= b` vaut déjà `True` ou `False` .

PARTIE C Méthode ou fonction ?

5 Expliquez pourquoi vous pouvez appeler `apple2.allumer()` mais que vous ne pouvez pas appeler `apple2.eteindre()` .

> **Coup de pouce** Regardez l'indentation de `eteindre` : à quel bloc appartient-elle ?

6 Comment faire pour appeler `eteindre()` ? Écrivez l'instruction qui éteint `apple2` .

7 Est-ce une bonne idée de mettre `eteindre` en dehors de la classe ? Pourquoi ?

Brouillon

EXERCICE 3 — ARPANET et le premier message 1969

Interaction entre objets et attributs complexes

Cet exercice porte sur la programmation orientée objet, les listes et leur parcours.

ARPANET est le précurseur d'Internet (1969). Il a permis la communication entre différents **hôtes** (ordinateurs). Chaque hôte est connecté au **réseau**, qui gère l'envoi des messages. Cet exercice simule des hôtes et un réseau qui interagissent.

Hote
`adresse_ip` `messages_recus`
`recevoir_message(message)` `lire_boite_reception()`

Reseau
`participants`
`connecter(hote)` `envoyer_message(emetteur_ip, destinataire_ip, message)`

PARTIE A Lire le code

On donne la classe `Hote` :

```
1  class Hote:
2      def __init__(self, adresse_ip):
3          self.adresse_ip = adresse_ip
4          self.messages_recus = []
5
6      def recevoir_message(self, message):
7          self.messages_recus.append(message)
8          print(f"Hôte {self.adresse_ip} a reçu : '{message[:10]}...'")
9
10     def lire_boite_reception(self):
11         return self.messages_recus
```

1 Dans la classe `Hote` , l'attribut `self.messages_recus` est une collection. Quel est son type ?

On donne la classe `Reseau`, dont la méthode `envoyer_message` est à compléter :

```
13 class Reseau:
14     def __init__(self):
15         self.participants = []
16
17     def connecter(self, hote):
18         self.participants.append(hote)
19         print(f"Réseau : Hôte {hote.adresse_ip} connecté.")
20
21     def envoyer_message(self, emetteur_ip, destinataire_ip, message):
22         print(f"Réseau : Tentative d'envoi de {emetteur_ip} vers {destinataire_ip}.")
23         # 1. parcours des hôtes connectés au réseau
24         for hote in _____:                                          ◂ question 4
25             # 2. recherche de l'hôte destinataire
26             if _____ == destinataire_ip:                            ◂ question 4
27                 # 3. Appeler la méthode de l'objet cible
28                 _____                                               ◂ question 4
29                 return True # Message envoyé
30         return False # Destinataire non trouvé
```

2 Dans la classe `Reseau`, l'attribut `self.participants` est une collection d'objets. Quel est le type des objets qu'elle contient ?

3 Si l'on ajoute 3 hôtes au réseau (`reseau.connecter(h1)`, puis `h2`, puis `h3`), quelle sera la longueur de `reseau.participants` ?

PARTIE B Compléter la méthode envoyer_message

4 Complétez le code de la méthode `envoyer_message` (lignes 24, 26 et 28).

> **Coup de pouce** Ligne 24 : quelle liste contient les hôtes connectés ? · Ligne 26 : comment lire l'adresse d'un hôte ? · Ligne 28 : quelle méthode de `Hote` permet de recevoir un message ?

5 Si le message est envoyé avec succès, quelle est l'action réalisée par le réseau sur l'objet destinataire ? (Nommez l'action et la méthode appelée.)

PARTIE C Tester

PROGRAMME DE TEST

```
1  reseau_arpa = Reseau()
2  h1 = Hote("192.168.1.1")
3  h2 = Hote("192.168.1.2")
4
5  reseau_arpa.connecter(h1)
6  reseau_arpa.connecter(h2)
7
8  reseau_arpa.envoyer_message("192.168.1.1", "192.168.1.2", "LOG=LOGIN")
9  reseau_arpa.envoyer_message("192.168.1.1", "192.168.1.3", "ERROR")
10
11 resultat = h2.lire_boite_reception()
```

6 Quel est le contenu final de la variable `resultat` ?

7 Pourquoi le deuxième appel à `reseau_arpa.envoyer_message()` renvoie-t-il `False` ?

EXERCICE 4

Le protocole TCP/IP de Vint Cerf et Bob Kahn 1974

Interaction de deux classes : écrire puis tracer le code

Cet exercice porte sur la programmation orientée objet : écriture de classes et interaction entre objets.

Vint Cerf et **Bob Kahn** ont mis au point le protocole TCP/IP, l'ensemble de règles qui régissent Internet, permettant aux ordinateurs de communiquer efficacement.

PARTIE A Écrire les classes Client et Serveur

Cahier des charges

Un `Client` doit pouvoir se connecter à un `Serveur`.

- La classe `Serveur` a un attribut `nb_clients_connectes` (initialisé à 0).
- La classe `Client` a un attribut `connecte_a` (initialisé à `None`).
- La méthode `se_connecter_a(self, serveur)` de la classe `Client` réalise deux actions :
  1. elle modifie son propre attribut `self.connecte_a` pour qu'il référence l'objet `serveur` ;
  2. elle appelle une méthode du `serveur` pour incrémenter son nombre de clients connectés.

Rappel de cours

`None` représente l'absence de valeur : au départ, le client n'est connecté à aucun serveur.

Un objet peut appeler la méthode d'un autre objet reçu en paramètre : `serveur.nom_methode()`.



1 Complétez la classe `Serveur` : son constructeur, puis une méthode qui ajoute 1 à `nb_clients_connectes` (choisissez-lui un nom explicite). Complétez ensuite la classe `Client` : son constructeur et la méthode `se_connecter_a`.

```
class Serveur:
    def __init__(self):
        ----------------------------------------------------------------------------------------------------
    def ------------------------------(self):
        ----------------------------------------------------------------------------------------------------


class Client:
    def __init__(self):
        ----------------------------------------------------------------------------------------------------

    def se_connecter_a(self, serveur):
        ------------------------------------------------------ # 1. référencer le serveur
        ------------------------------------------------------ # 2. prévenir le serveur
```

Brouillon

PARTIE B Synthèse et tracé du code

On exécute le programme ci-dessous en utilisant les classes écrites dans la partie A.

PROGRAMME PRINCIPAL

```
1  s1 = Serveur()
2  c1 = Client()
3  c2 = Client()
4
5  c1.se_connecter_a(s1)
6  c2.se_connecter_a(s1)
7  c3 = Client()
8  c3.se_connecter_a(s1)
9
10 nb = s1.nb_clients_connectes
11 type_c1 = type(c1.connecte_a)
```

1 Quel est le nom de la méthode que la ligne `c1.se_connecter_a(s1)` appelle sur l'objet `s1` ?

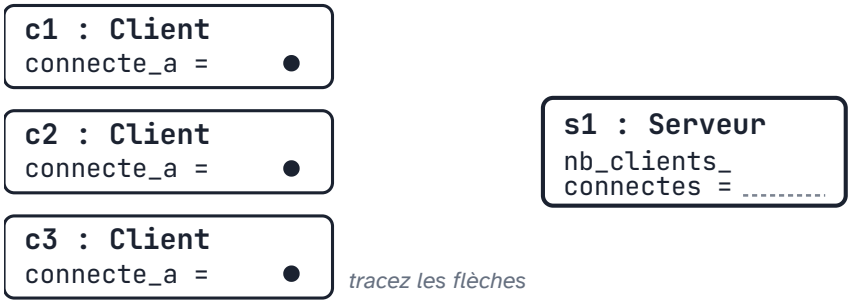
2 Quelle est la valeur finale de la variable `nb` ? Suivez son évolution dans le tableau.

Après la ligne	1	5	6	8
`s1.nb_clients_connectes`	0			

3 Que contient la variable `type_c1` ? Autrement dit, de quel type est `c1.connecte_a` ?

4 Les attributs `c1.connecte_a` et `c2.connecte_a` référencent-ils le même objet `Serveur` ? Justifiez.

> **Coup de pouce** Complétez le schéma mémoire : une flèche part de chaque attribut `connecte_a` vers l'objet qu'il référence.



tracez les flèches

Brouillon

Bilan — je sais…

- ☐ Je prédis l'affichage d'un constructeur lors d'une instanciation.
- ☐ J'écris une méthode qui **modifie** un attribut ou qui **renvoie** une valeur.
- ☐ Je simule une boucle qui appelle une méthode.
- ☐ Je parcours une liste d'objets et j'appelle leurs méthodes.
- ☐ Je fais interagir deux objets (un objet en attribut d'un autre).
- ☐ Je distingue une **méthode** d'une **fonction**.